

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ MYOS

Chương này trình bày quá trình chuyển các mục tiêu của đề tài thành mô hình kiến trúc và các cơ chế thiết kế cụ thể cho MyOS. Trọng tâm của chương là xác định hệ thống cần cung cấp những chức năng nào, các thành phần phối hợp với nhau ra sao và những nguyên tắc được lựa chọn để phù hợp với nền tảng ARM Cortex-M4F. Các chi tiết hiện thực bằng tệp mã nguồn, hàm và câu lệnh được trình bày trong Chương 4.

3.1. Phân tích yêu cầu hệ thống

MyOS được định hướng là một RTOS nhúng quy mô nhỏ phục vụ nghiên cứu và thực nghiệm trên STM32F407. Hệ thống phải hỗ trợ nhiều task cùng hoạt động, ưu tiên các công việc quan trọng, cho phép task chờ sự kiện mà không chiếm CPU và cung cấp các cơ chế quan sát cần thiết để kiểm tra hoạt động của kernel.

Các yêu cầu chức năng chính được xác định như sau:

- Quản lý vòng đời task, gồm tạo, thực thi, chờ, tạm dừng, tiếp tục và kết thúc.
- Lập lịch theo độ ưu tiên, hỗ trợ cưỡng bức và chia sẻ CPU giữa các task cùng ưu tiên.
- Cung cấp nguồn thời gian cho delay, timeout và software timer.
- Hỗ trợ đồng bộ hóa bằng semaphore và mutex; hỗ trợ giao tiếp bằng queue, message buffer và event group.
- Quản lý heap và stack, phát hiện bất thường của stack và cung cấp thống kê bộ nhớ.
- Khai thác SVC, PendSV, SysTick, NVIC, MPU và FPU của Cortex-M4F.
- Cung cấp trace, runtime statistics, fault handling, UART shell và chế độ tiết kiệm năng lượng.

Về yêu cầu phi chức năng, kernel cần có cấu trúc rõ ràng, giới hạn tài nguyên có thể cấu hình, thời gian xử lý ngắn và hành vi có thể dự đoán. Thiết kế phải hạn chế phụ thuộc phần cứng trong phần lõi, sử dụng critical section để bảo vệ dữ liệu dùng chung và cho phép kiểm thử từng nhóm chức năng độc lập.

Nhóm yêu cầu	Mục tiêu thiết kế
Tính đúng đắn	Duy trì nhất quán trạng thái task, danh sách chờ và tài nguyên kernel.
Tính thời gian thực	Ưu tiên task quan trọng và giảm công việc trong ngữ cảnh ngắt.
Tính mô-đun	Tách kernel, architecture port, board, driver, HAL và application.
Khả năng quan sát	Theo dõi task, stack, heap, trace và thời gian sử dụng CPU.
Khả năng mở rộng	Cho phép bổ sung driver, thuật toán lập lịch và nền tảng Cortex-M khác.

3.2. Kiến trúc tổng thể

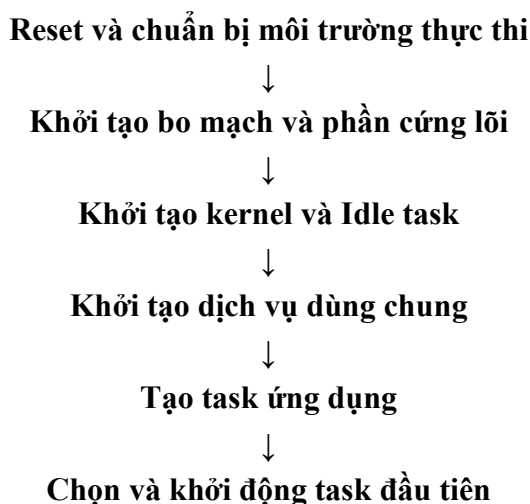
MyOS được thiết kế theo kiến trúc phân lớp. Mỗi lớp đảm nhận một nhóm trách nhiệm và chỉ nên phụ thuộc vào các giao diện của lớp thấp hơn. Cách tổ chức này giúp tách phần logic RTOS khỏi chi tiết của vi điều khiển và giảm ảnh hưởng khi thay đổi phần cứng.

Lớp	Vai trò thiết kế
Application	Chứa task ứng dụng, tài nguyên dùng chung và kịch bản kiểm thử.
Kernel	Quản lý task, scheduler, thời gian, đồng bộ, IPC, bộ nhớ và giám sát.
Architecture port	Cung cấp chuyển ngữ cảnh và thao tác phụ thuộc Cortex-M4F.
HAL và driver	Trừu tượng hóa và điều khiển ngoại vi.
Board và platform	Khởi tạo bo mạch, startup, sơ đồ bộ nhớ và cấu hình nền tảng.
CMSIS	Cung cấp định nghĩa chuẩn cho lõi Cortex-M4 và STM32F407.

Kernel được thiết kế để không thao tác trực tiếp với thanh ghi phần cứng, ngoại trừ thông qua architecture port hoặc các giao diện nền tảng. Application sử dụng dịch vụ kernel và HAL thay vì can thiệp vào cấu trúc quản lý nội bộ. Đây là nguyên tắc chính để bảo đảm khả năng bảo trì và tái sử dụng.

3.3. Thiết kế quá trình khởi động

Quá trình khởi động được chia thành các giai đoạn có thứ tự rõ ràng để bảo đảm mỗi thành phần chỉ được sử dụng sau khi các phụ thuộc của nó đã sẵn sàng. Trước tiên, môi trường thực thi và phần cứng cơ bản được thiết lập. Sau đó kernel khởi tạo các cấu trúc quản lý, ứng dụng tạo tài nguyên và task, cuối cùng scheduler tiếp nhận quyền điều khiển.

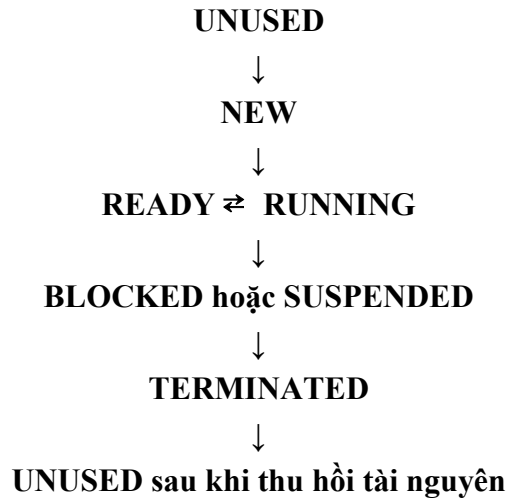


Thiết kế này tránh việc ngắt hoặc scheduler hoạt động khi bảng task, heap và các danh sách quản lý chưa hoàn chỉnh. Sau khi task đầu tiên được khởi động, luồng thực thi không quay lại chương trình khởi tạo mà được điều phối bởi scheduler và các exception của kernel.

3.4. Thiết kế quản lý task

Mỗi task được biểu diễn bởi một TCB chứa thông tin nhận dạng, trạng thái, độ ưu tiên, ngưỡng thực thi, stack, thời gian chờ, vùng MPU và các tài nguyên đồng bộ đang liên quan. Các TCB được quản lý trong một bảng có kích thước giới hạn trước, phù hợp với tài nguyên RAM của vi điều khiển.

Vòng đời task gồm các trạng thái Unused, New, Ready, Running, Blocked, Suspended và Terminated. Task mới được chuẩn bị ngưỡng cảnh rồi chuyển vào Ready. Scheduler chuyển task được chọn sang Running. Task chờ thời gian, sự kiện hoặc tài nguyên chuyển sang Blocked; task bị tạm dừng chuyển sang Suspended; task kết thúc chuyển sang Terminated trước khi tài nguyên được thu hồi.

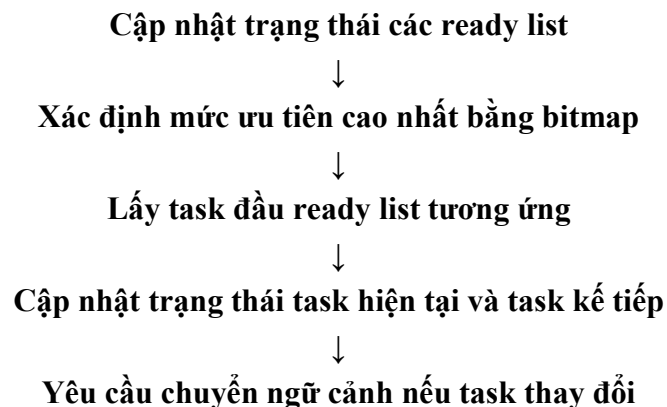


MyOS hỗ trợ stack cấp phát động và stack do ứng dụng cung cấp. Stack cần được căn chỉnh phù hợp với yêu cầu của CPU và MPU. Việc thu hồi stack của task đang chạy được trì hoãn và giao cho Idle task để tránh giải phóng vùng nhớ mà CPU vẫn đang sử dụng.

3.5. Thiết kế scheduler

Scheduler được thiết kế theo mô hình ưu tiên cố định có cưỡng bức. Mỗi mức ưu tiên có một ready list riêng; một bitmap cho biết nhanh những mức ưu tiên đang có task sẵn sàng. Scheduler luôn chọn task thuộc mức ưu tiên cao nhất.

Khi một task ưu tiên cao hơn chuyển sang Ready, hệ thống có thể yêu cầu lập lịch lại ngay. Với các task cùng ưu tiên, round-robin kết hợp time slicing được sử dụng để phân phối CPU luân phiên. Idle task giữ mức ưu tiên thấp nhất và chỉ chạy khi không còn task ứng dụng sẵn sàng.

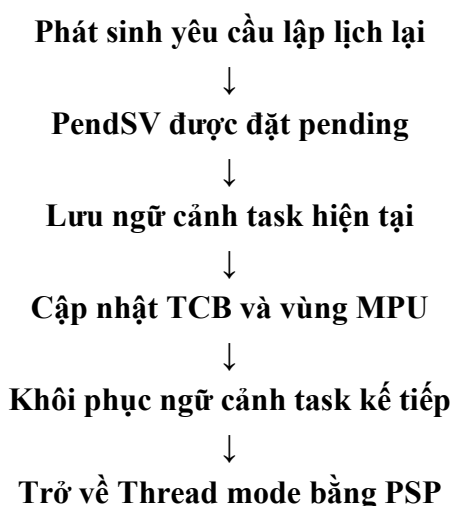


Thiết kế ready list theo từng ưu tiên giúp thao tác lựa chọn task không phải duyệt toàn bộ bảng TCB. Scheduler cũng có cơ chế khóa tạm thời để trì hoãn việc lập lịch trong các đoạn cập nhật dữ liệu kernel cần tính nguyên tử.

3.6. Thiết kế chuyển ngữ cảnh

Mỗi task sử dụng Process Stack Pointer, trong khi kernel và exception sử dụng cơ chế stack của Cortex-M. Khi exception xảy ra, phần cứng tự động lưu nhóm thanh ghi cơ bản. Phần mềm chỉ cần bổ sung các thanh ghi còn lại và lưu con trỏ stack vào TCB của task hiện tại.

PendSV được chọn làm exception thực hiện chuyển ngữ cảnh vì có thể đặt ở mức ưu tiên thấp. Các xử lý ngắt khác chỉ yêu cầu lập lịch lại; việc lưu và khôi phục ngữ cảnh được trì hoãn cho đến khi các ngắt quan trọng hơn hoàn tất. Nếu task sử dụng FPU, phần ngữ cảnh dấu phẩy động mở rộng cũng phải được bảo toàn.



3.7. Thiết kế hệ thống tick

SysTick được sử dụng làm nguồn thời gian định kỳ của kernel. Mỗi tick làm tăng thời gian hệ thống, cập nhật software timer, kiểm tra task hết thời gian chờ và xử lý time slice của task đang chạy.

Thông tin timeout được lưu theo thời điểm hết hạn thay vì giảm một bộ đếm riêng cho từng API. Khi thời hạn đạt tới, task được tách khỏi danh sách chờ, nhận kết quả timeout và trở lại Ready. Các phép so sánh thời gian cần được thiết kế để vẫn đúng khi bộ đếm tick tràn.

Trình xử lý tick chỉ thực hiện công việc cần thiết và yêu cầu lập lịch lại khi có task ưu tiên cao hơn được đánh thức hoặc time slice kết thúc. Việc chuyển ngữ cảnh thực tế vẫn được giao cho PendSV.

3.8. Thiết kế hệ thống ngắt và exception

MyOS phân biệt rõ ngữ cảnh task và ngữ cảnh ngắt. API thông thường có thể block nên không được gọi trong ISR. Những thao tác cần dùng trong ISR phải có biến thể không block, thời gian xử lý ngắn và chỉ yêu cầu PendSV khi cần đánh thức task ưu tiên cao hơn.

Thành phần	Vai trò trong thiết kế
SVC	Chuyển có kiểm soát vào Handler mode và khởi động task đầu tiên.
PendSV	Thực hiện chuyển ngữ cảnh ở mức ưu tiên thấp.
SysTick	Cung cấp tick định kỳ và kích hoạt xử lý thời gian.
Ngắt ngoại vi	Tiếp nhận sự kiện phần cứng và đánh thức task xử lý.
Fault exception	Thu thập trạng thái lỗi và đưa hệ thống về trạng thái an toàn.

Thứ tự ưu tiên được lựa chọn sao cho ngắt ngoại vi quan trọng có thể hoàn tất trước, SysTick cập nhật thời gian, còn PendSV chỉ chạy khi không còn exception ưu tiên cao hơn. Thiết kế này giảm nguy cơ chuyển ngữ cảnh giữa lúc ISR đang cập nhật tài nguyên kernel.

3.9. Thiết kế SCB, NVIC và FPU

SCB được sử dụng để cấu hình bảng vector, cho phép các fault có thể cấu hình và thiết lập hành vi của hệ thống khi xảy ra lỗi. NVIC cung cấp cơ chế bật, tắt, đặt pending và phân mức ưu tiên cho ngắt. Các thao tác này được đặt sau một giao diện chung để giảm phụ thuộc trực tiếp của kernel vào thanh ghi.

FPU được bật cho phép sử dụng phép toán dấu phẩy động phần cứng. Thiết kế chuyển ngữ cảnh dựa vào trạng thái exception return để xác định task có ngữ cảnh FPU hay không, từ đó chỉ lưu nhóm thanh ghi mở rộng khi cần nhằm giảm chi phí cho các task không sử dụng dấu phẩy động.

3.10. Thiết kế đồng bộ hóa

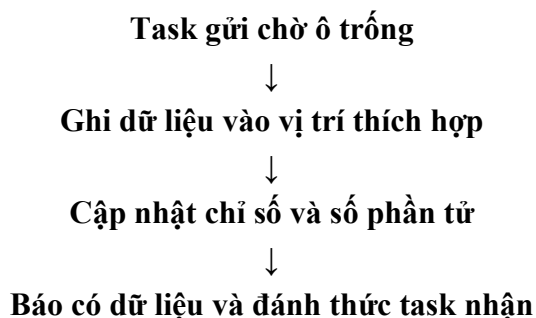
Semaphore được thiết kế dưới dạng bộ đếm kèm giới hạn tối đa và một wait list. Binary semaphore biểu diễn sự kiện hai trạng thái, trong khi counting semaphore phù hợp với số lượng tài nguyên hoặc số sự kiện chưa xử lý. Khi không thể lấy semaphore, task có thể block vô thời hạn hoặc chờ trong một khoảng timeout.

Mutex bổ sung khái niệm task sở hữu và số lần khóa. Recursive mutex cho phép chính task sở hữu khóa nhiều lần và chỉ giải phóng hoàn toàn khi số lần mở khóa tương ứng. Task không sở hữu mutex không được phép giải phóng nó.

Để hạn chế priority inversion, khi task ưu tiên cao chờ mutex do task ưu tiên thấp giữ, task sở hữu tạm thời kế thừa mức ưu tiên cao hơn. Khi mutex được giải phóng, mức ưu tiên hiệu dụng được tính lại dựa trên ưu tiên cơ sở và các mutex còn giữ. Task được đánh thức từ wait list theo ưu tiên thay vì chỉ theo thứ tự đến.

3.11. Thiết kế IPC

Message queue được thiết kế dưới dạng bộ đệm có số phần tử và kích thước phần tử xác định. Hai cơ chế đếm được dùng để biểu diễn số ô có dữ liệu và số ô còn trống. Task gửi sẽ block khi queue đầy, còn task nhận sẽ block khi queue rỗng, trừ khi timeout bằng không.



Message buffer được xây dựng để truyền dữ liệu có kích thước thay đổi thông qua queue mà không làm thay đổi cấu trúc phần tử của queue. Quyền sở hữu vùng dữ liệu phải được chuyển rõ ràng giữa bên gửi và bên nhận để tránh rò rỉ hoặc giải phóng hai lần.

Các thao tác IPC từ ISR được thiết kế không block. ISR chỉ gửi hoặc nhận khi trạng thái queue cho phép và yêu cầu lập lịch lại nếu thao tác làm một task ưu tiên cao hơn sẵn sàng.

3.12. Thiết kế event group

Event group sử dụng một biến bit để biểu diễn nhiều sự kiện độc lập trong cùng một đối tượng. Task có thể chờ một trong các bit hoặc chờ đồng thời tất cả các bit được chỉ định. Tùy lựa chọn, những bit thỏa điều kiện có thể được xóa tự động khi task rời trạng thái chờ.

Khi bit sự kiện được đặt, hệ thống duyệt các task đang chờ, kiểm tra điều kiện của từng task và đưa các task thỏa mãn về Ready. Kết quả bit tại thời điểm đánh thức được lưu riêng cho task để không bị thay đổi bởi các thao tác tiếp theo. Thao tác đặt bit từ ISR tuân theo nguyên tắc không block.

3.13. Thiết kế quản lý bộ nhớ

Heap được thiết kế theo danh sách các block trống. Khi cấp phát, hệ thống chọn một block đủ lớn, tách phần dư nếu cần và đánh dấu block đã sử dụng. Khi giải phóng, các block trống liền kề được gộp lại nhằm hạn chế phân mảnh ngoài.

Bộ quản lý heap theo dõi tổng dung lượng còn trống, mức trống thấp nhất từng ghi nhận, block trống lớn nhất và số block trống. Tỷ lệ phân mảnh được ước lượng từ quan hệ giữa block lớn nhất và tổng vùng trống. Các thao tác thay đổi danh sách block phải được bảo vệ khỏi truy cập đồng thời.

Mỗi task có stack riêng. Vùng stack được điền bằng một mẫu xác định để đo high-water mark và đặt guard words ở đáy để phát hiện ghi tràn. Con trỏ stack cũng phải nằm trong phạm vi đã cấp phát. Cơ chế này hỗ trợ phát hiện lỗi sớm và lựa chọn kích thước stack phù hợp.

3.14. Thiết kế MPU

MPU được thiết kế với hai nhóm vùng. Các vùng tĩnh mô tả code, RAM và ngoại vi dùng chung của hệ thống; các vùng động thay đổi theo task để bảo vệ stack và một vùng dữ liệu mở rộng. Các vùng dữ liệu được đặt thuộc tính không thực thi nhằm giảm nguy cơ chạy mã từ RAM ngoài dự kiến.

Kích thước và địa chỉ vùng MPU phải đáp ứng yêu cầu lũy thừa của hai và căn chỉnh theo kích thước vùng. Vì vậy, thiết kế cấp phát stack cần phối hợp với thiết kế MPU. Cấu hình vùng động được cập nhật khi chuyển task trước khi CPU trở lại Thread mode.

Trong phạm vi hiện tại, MPU tạo nền tảng cho bảo vệ bộ nhớ nhưng chưa hình thành mô hình user mode hoàn chỉnh, vì task vẫn chạy đặc quyền. Thiết kế được chuẩn bị để mở rộng sang task không đặc quyền và system call thông qua SVC.

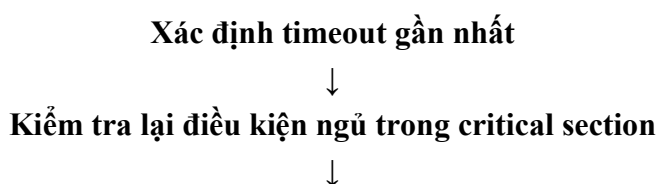
3.15. Thiết kế software timer

Software timer được mô hình hóa bằng thời điểm hết hạn, chu kỳ, chế độ hoạt động và hàm callback. Hệ thống hỗ trợ timer một lần và timer định kỳ. Các timer đang hoạt động được tổ chức theo thời điểm hết hạn để giảm số đối tượng cần kiểm tra tại mỗi tick.

Khi timer hết hạn, callback được thực thi và timer định kỳ được lập lịch lại theo chu kỳ đã cấu hình. Thiết kế phải xử lý trường hợp callback dừng hoặc khởi động lại chính timer đang chạy, đồng thời tránh làm hỏng danh sách timer.

3.16. Thiết kế tickless idle

Tickless idle nhằm giảm số lần CPU thức dậy khi hệ thống không có công việc. Chế độ này chỉ được xem xét khi Idle task đang chạy, không có task Ready khác và ứng dụng không đặt điều kiện cấm ngủ.



Lập trình khoảng ngủ dự kiến



Đưa CPU vào trạng thái chờ



Tính số tick thực tế đã ngủ



Bù thời gian và xử lý các timeout đã đến hạn

Khoảng ngủ phải bị giới hạn bởi khả năng đếm của SysTick. Sau khi thức dậy, kernel cập nhật thời gian hệ thống, software timer và task timeout trước khi khôi phục tick định kỳ. Thiết kế cần chấp nhận khả năng CPU thức sớm do một nguồn ngắt khác.

3.17. Thiết kế giám sát và xử lý lỗi

Khả năng quan sát được xem là một phần của thiết kế kernel. Trace buffer lưu vòng các sự kiện quan trọng như tạo task, block, wakeup, chuyển task, semaphore, mutex, queue và low-power. Mỗi bản ghi chứa thời gian, task liên quan và tham số hỗ trợ phân tích.

Runtime statistics tích lũy thời gian CPU của từng task tại các điểm chuyển ngữ cảnh. Thông tin stack và heap cung cấp góc nhìn về mức sử dụng bộ nhớ. Các kiểm tra assert bảo vệ giả định nội bộ của kernel trong giai đoạn phát triển.

Khi xảy ra fault hoặc phát hiện lỗi nghiêm trọng như hỏng heap hay tràn stack, hệ thống thu thập trạng thái CPU và các thanh ghi lỗi, xuất thông tin chẩn đoán qua kênh an toàn rồi dừng để tránh tiếp tục vận hành với trạng thái không xác định.

3.18. Thiết kế UART shell

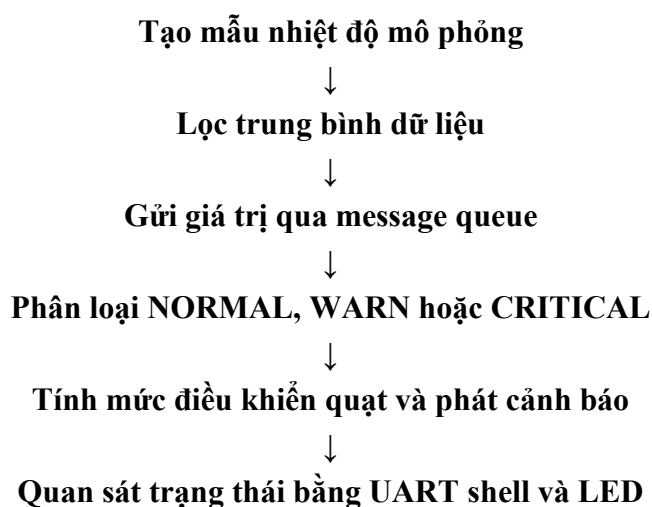
UART shell được thiết kế như một task quan sát và điều khiển hệ thống trong thời gian chạy. Shell tiếp nhận chuỗi lệnh, phân tích tham số và gọi các giao diện công khai thay vì thay đổi trực tiếp cấu trúc nội bộ của kernel.

Các nhóm lệnh gồm xem danh sách task, runtime statistics, trace, heap, stack, queue, trạng thái năng lượng và ứng dụng minh họa; ngoài ra có thể tạm dừng, tiếp tục, kết thúc task hoặc khởi động lại hệ thống. Dữ liệu in ra UART được bảo vệ để tránh nhiều task ghi xen kẽ.

Shell chủ yếu phục vụ phát triển và kiểm thử. Các thao tác in đồng bộ có thể ảnh hưởng đến thời gian thực, do đó không nên thực hiện trong ISR hoặc trong critical section kéo dài.

3.19. Thiết kế ứng dụng minh họa

Ứng dụng minh họa được xây dựng theo bài toán giám sát nhiệt độ nhằm thể hiện sự phối hợp giữa task, queue, mutex, shell và GPIO. Dữ liệu nhiệt độ hiện được tạo bởi cảm biến mô phỏng, sau đó được lọc trước khi chuyển đến bộ điều khiển.



Task cảm biến hoạt động định kỳ và gửi dữ liệu qua queue. Task điều khiển chờ dữ liệu, xác định vùng nhiệt độ và cập nhật mức điều khiển. Mutex bảo vệ trạng thái tổng hợp khi nhiều task cùng truy cập. Task shell cung cấp thông tin hệ thống, còn task GPIO tạo tín hiệu quan sát trực quan.

Ứng dụng được giữ ở mức đơn giản để trọng tâm vẫn là các dịch vụ RTOS. Thiết kế cho phép thay nguồn dữ liệu mô phỏng bằng cảm biến thực trong giai đoạn mở rộng mà không thay đổi các cơ chế quản lý task và IPC của kernel.

3.20. Kết chương

Chương 3 đã xác định các yêu cầu và thiết kế tổng thể của MyOS, từ kiến trúc phân lớp đến quản lý task, scheduler, chuyển ngữ cảnh, thời gian, đồng bộ, IPC, bộ nhớ, MPU, giám sát và ứng dụng minh họa. Các thiết kế được lựa chọn theo hướng nhỏ gọn, có thể quan sát và phù hợp với tài nguyên của STM32F407.

Trên cơ sở đó, Chương 4 trình bày cách các thành phần được hiện thực bằng mã nguồn, cách tổ chức các module và luồng vận hành thực tế của hệ thống.